./

WATCHDOG

2.4 GHz Wireless Security Analyser

Pattern Detection Engine — v3.1 ENGINEERING HARDENED

ESP32 + nRF24L01 + ILI9341 TFT
2.4 GHz RF Spectrum Analyser
Jammer Detection · WiFi Scanner · BLE Mapper
Camera OUI Scanner · Tracker Detection
RF Foxhunt · Lens Scanner · Geiger Counter
Wireless IDS · Touchscreen UI · ECO / Stealth Mode
FreeRTOS BLE Mutex · Touch Wakeup from Deep Sleep
Heap-Safe char* Stack · Capped Scale Multiplier

Author Anirudh N.S.

Version v3.1 ENGINEERING HARDENED

Architecture Single-Loop, Async Nets, Deferred-Action, FreeRTOS Mutex

Status Production — Hardware Tested

Copyright © 2025 Anirudh N.S. — All Rights Reserved

Personal cybersecurity project.
All detection is passive RF energy sensing — no transmission.

Contents

1 Project Overview **3**

2 Hardware Gallery — Field Photographs **3**

3 Hardware **7**
3.1 Bill of Materials . . . 7
3.2 Pin Assignment . . . 7
3.3 SPI Bus Architecture . . . 9
3.4 Power System — TP4056 LiPo Charge Module . . . 9
3.5 Physical Build — 7×9 cm Perfboard . . . 10

4 Software Architecture **10**
4.1 Execution Model . . . 10
4.2 FreeRTOS BLE Mutex . . . 11
4.2.1 The Race Condition in v3.0 . . . 11
4.2.2 v3.1 Fix: Binary Mutex . . . 11
4.2.3 Mutex-Protected Sites . . . 12
4.3 Deferred-Action Queue . . . 12
4.4 Timing Budget . . . 12
4.5 Library Dependencies . . . 13
4.6 Key Data Structures . . . 13
4.7 Named Colour Constants . . . 14

5 RF Detection Engine **14**
5.1 Sensing Method . . . 14
5.2 Early Exit and Instant Jam Triggers . . . 14
5.3 earlyActive Per-Pass Reset — v3.1 Fix . . . 15
5.4 Scale Multiplier Cap — v3.1 Fix . . . 15
5.5 Exponential Smoothing Filter . . . 16
5.6 Adaptive Noise Floor . . . 16
5.7 Feature Extraction . . . 16
5.8 Shannon Entropy Analysis — v3.1 Enhancement . . . 16
5.9 Separated Hop and Sweep Detection Pipelines . . . 17
5.10 Pattern Classification — Priority-Ordered Gates . . . 17
5.11 Persistence Counters, Final Priority, and 1-Second Hold-Off . . . 18
5.12 Jammer Type Reference . . . 18
5.13 Stability Buffer, Lens Scanner, and Overflow Caps . . . 19

6 User Interface **19**
6.1 Display Architecture . . . 19
6.2 Long-Press Sub-Mode System . . . 19
6.3 Pages . . . 20
6.3.1 Page 0 — HOME . . . 20
6.3.2 Page 1 — RF SPECTRUM . . . 20
6.3.3 Page 2 — WiFi SCANNER . . . 20
6.3.4 Page 3 — BLUETOOTH MAPPER . . . 20
6.3.5 Page 4 — WIRELESS IDS . . . 20
6.3.6 Page 5 — SYSTEM STATUS . . . 20

6.4 Status Bar . . . 20
6.5 Backlight Management . . . 21

7 Alarm System **21**
7.1 Non-Blocking Buzzer Architecture . . . 21
7.2 jamLabel() — v3.1 Fix: const char* vs String . . . 21
7.3 totalAlerts Overflow Protection — v3.1 Fix . . . 21
7.4 Tone Patterns by Jam Type . . . 22
7.5 Alarm Cadence by Threat Level . . . 22
7.6 Geiger Counter Mode . . . 22

8 WiFi and BLE Scanning **23**
8.1 Asynchronous WiFi Scan with Cooldown Guard . . . 23
8.2 Camera OUI Detection — v3.1 Expanded Database . . . 23
8.3 Evil Twin Detection . . . 23
8.4 Active BLE Scan — v3.1 Fix . . . 23
8.5 Async BLE Scan Trigger with Mutex Guard . . . 24
8.6 BLE Tracker Detection — v3.1 Expanded Database . . . 24

9 Power Management **25**
9.1 Virtual Battery Model . . . 25
9.2 ECO / Battery Saver Mode . . . 25
9.3 Idle Auto-Sleep with Touch Wakeup — v3.1 . . . 25

10 Stability Engineering **26**
10.1 Non-Lethal Health Monitor . . . 27

11 Testing and Calibration **27**
11.1 The RF Interference Source . . . 28
11.2 Observed Signal Characteristics . . . 28
11.3 v3.1 False Positive Regression — earlyActive Fix . . . 28
11.4 Serial Monitor Output . . . 28

12 Installation and Build **29**
12.1 Hardware Assembly . . . 29
12.2 Firmware Upload . . . 29
12.3 First Boot Verification . . . 29
12.4 nRF24 Hardware Fault Path . . . 29

13 Known Limitations **30**

14 Roadmap **31**

15 Changelog **31**

16 Copyright **33**

1 Project Overview

WatchDog 2.4 is a battery-powered, handheld wireless security analyser built around the ESP32 microcontroller. It passively monitors the 2.4 GHz radio band — the frequency range shared by Wi-Fi, Bluetooth, BLE, RC drones, and many other wireless protocols — and detects the presence of radio frequency jamming attacks in real time.

The device uses an nRF24L01 radio module as a wideband energy sensor, sweeping all 125 channels of the 2.4 GHz spectrum in rapid succession and feeding the raw energy readings through an exponential smoothing filter and a multi-stage pattern classifier. This engineering hardened build (`v3.1 ENGINEERING HARDENED`) resolves two critical race conditions present in v3.0, hardens the RF scanning pipeline against false positives, extends the device OUI databases, and fixes several medium and low-priority issues identified during extended field use.

- Sweeps 125 RF channels across 2.400–2.525 GHz
- Classifies jamming: RC, BT, BLE, Wi-Fi, Deauth Flood, Drone FHSS, Sweep Jammer, Broadband, High-Entropy Anomaly
- Scans nearby Wi-Fi networks, detects open APs, evil twins, and known camera OUIs (7-entry database)
- Maps nearby BLE devices; flags trackers (AirTag, Tile, SmartTag, FindMy, Chipolo) and hidden-MAC proximity devices
- Runs a Wireless IDS with five persistence counters and a live alert log
- RF Foxhunt mode with Geiger-counter audio proximity feedback
- Lens Scanner mode for detecting hidden camera reflections from RF stability patterns
- ECO battery-saver mode with deferred theme enforcement and scan suspension
- Light / Dark UI theme switching via deferred-action queue
- Triggers a buzzer alarm with unique tones per threat type
- Displays a live 125-channel spectrum analyser with peak-hold and colour gradient
- Deep sleep with touch-wakeup — no physical power-cycle required to resume

WatchDog 2.4 is a **passive listening device only**. It does not transmit any radio signal. It is designed for legitimate security research, education, and portfolio demonstration. Deploying RF jammers is illegal in most jurisdictions. This device detects them — it does not replicate them.

2 Hardware Gallery — Field Photographs

The photographs below show the physical WatchDog 2.4 device operating in a live RF environment. The build consists of an ESP32 DevKit, nRF24L01+ PA/LNA module with external antenna, ILI9341 TFT with XPT2046 touch controller, active buzzer, and 100 µF decoupling capacitor, all hand-wired on a 7×9 cm perfboard and powered via a TP4056 LiPo charge module.

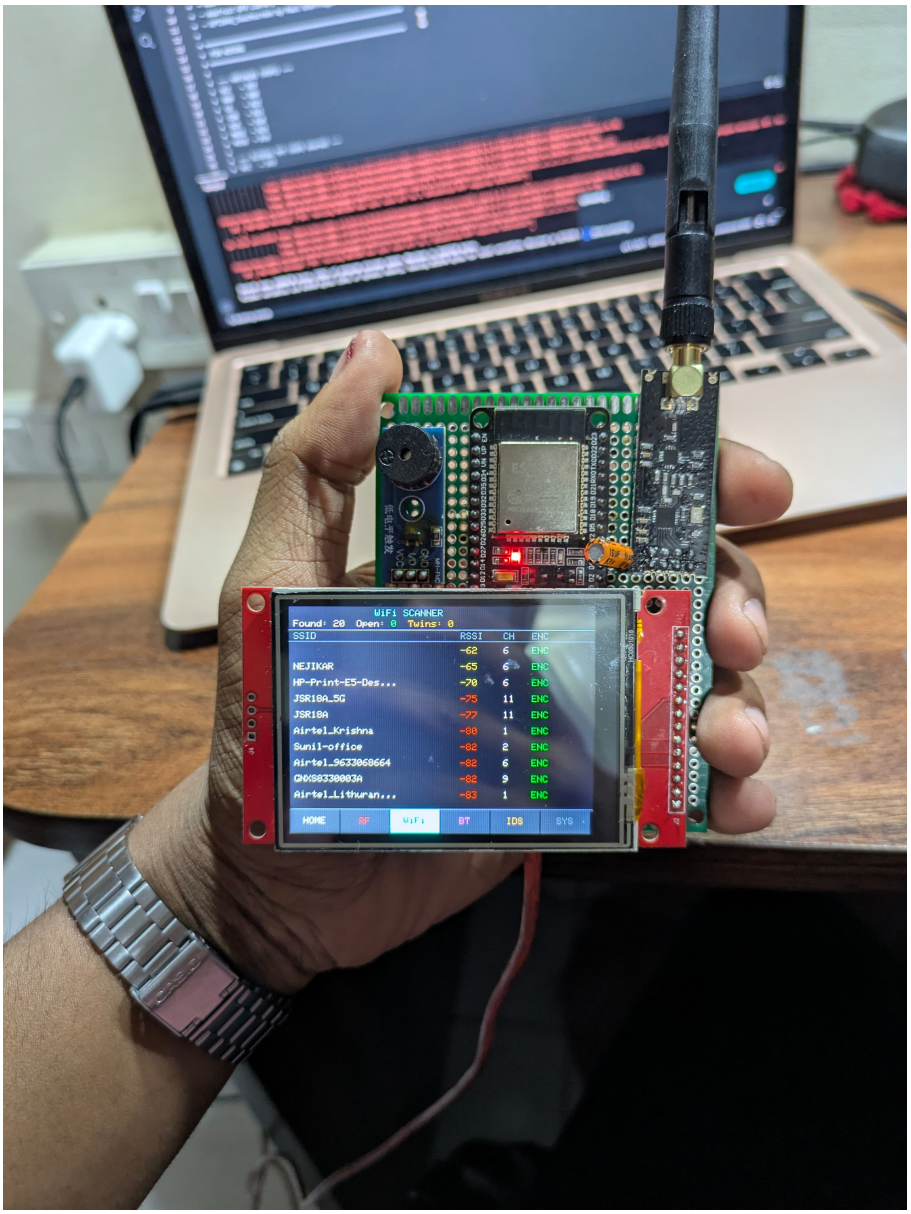


Figure 1: **WiFi Scanner page — live environment.** WatchDog 2.4 capturing 20 Wi-Fi networks in a residential building. All detected networks report encrypted status (ENC); no open access points or evil twin SSIDs were found in this scan. RSSI values span −62 to −83 dBm across channels 1, 2, 6, 9, and 11. The nRF24L01+ PA/LNA module with its external stub antenna is visible above the perfboard assembly.



Figure 2: **Bluetooth Mapper page — 7 BLE devices in range.** The device at −27 dBm (bottom row) registers a full 5-bar green signal strength, indicating a nearby transmitter. Several devices advertise only their MAC address and no friendly name — flagged as suspicious by the unnamed-device heuristic (`!d.haveName() && d.getRSSI() > -65`). The 100 µF nRF24L01 decoupling capacitor is visible in amber on the perfboard. The ESP32 module label is clearly legible in the background.

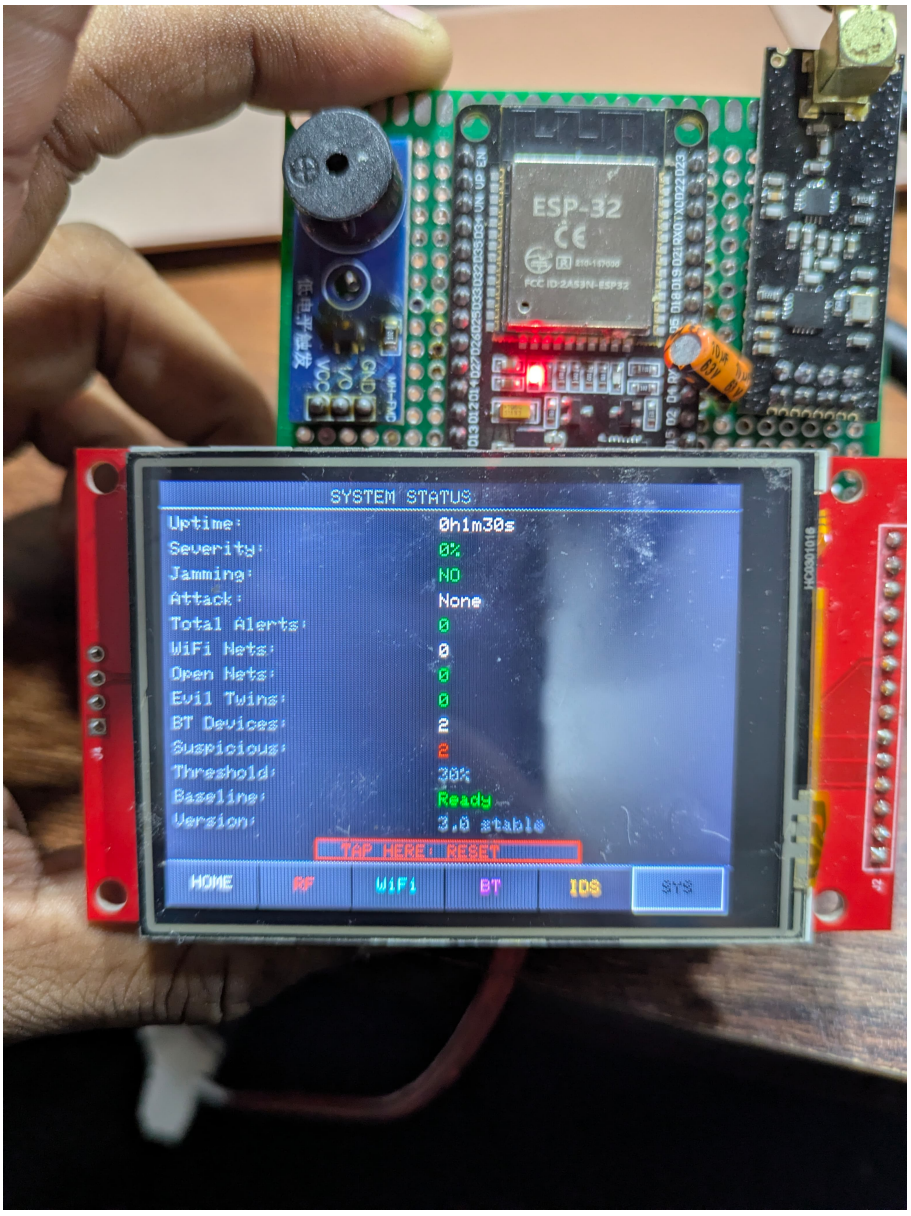


Figure 3: **System Status page — 1 min 30 s uptime.** Jamming state: NO. Two Bluetooth devices detected, both flagged suspicious (count highlighted in red). No RF alerts, open networks, or evil twins recorded in this session. The noise floor baseline is calibrated and ready. All diagnostic counters visible: uptime, severity percentage, attack type, total alerts, WiFi net count, open nets, evil twins, BT devices, and suspicious device count.

3 Hardware

3.1 Bill of Materials

Component	Qty	Notes
ESP32 DevKit v1 (30-pin)	1	Dual-core 240 MHz, built-in Wi-Fi + BT
nRF24L01+ with PA/LNA	1	External antenna model recommended
ILI9341 320×240 TFT	1	With XPT2046 resistive touch controller
Active buzzer (3-pin)	1	3.3V compatible, `tone()`-driven
100 µF capacitor	1	Across nRF24 VCC/GND — mandatory
3.7V LiPo battery	1	1000–2000 mAh
TP4056 charge module	1	USB-C variant preferred
Slide switch	1	In-line on OUT+ rail from TP4056

The nRF24L01 module is extremely sensitive to power supply noise. Always place a 100 µF electrolytic capacitor between VCC and GND as close to the module as possible. Omitting this causes missed detections, false positives, and random resets.

3.2 Pin Assignment

Signal	ESP32 Pin	GPIO	Notes
nRF24L01 Radio			
SPI SCK	D18	GPIO18	Shared SPI bus
SPI MOSI	D23	GPIO23	Shared SPI bus
SPI MISO	D19	GPIO19	Shared SPI bus
CE	D22	GPIO22	Chip Enable
CSN	D21	GPIO21	Chip Select
VCC	3V3	—	With 100 µF cap
GND	GND	—	
ILI9341 TFT Display			
TFT CS	D5	GPIO5	
TFT DC	D32	GPIO32	Data/Command
TFT RST	D33	GPIO33	Reset
TFT SCK	D18	GPIO18	Shared SPI
TFT MOSI	D23	GPIO23	Shared SPI
TFT MISO	D19	GPIO19	Shared SPI
TFT BL	D27	GPIO27	Backlight PWM (`ledc`)

Signal	ESP32 Pin	GPIO	Notes
XPT2046 Touch Controller			
T_CS	D13	GPIO13	Touch Chip Select
T_IRQ	D4	GPIO4	Touch Interrupt — **also deep-sleep wakeup source (v3.1)**
T_CLK	D18	GPIO18	Shared SPI
T_DIN	D23	GPIO23	Shared SPI
T_OUT	D19	GPIO19	Shared SPI
Buzzer			
Signal	D26	GPIO26	`tone()` output
VCC	3V3	—	
GND	GND	—	
Battery ADC (Reserved)			
BAT_ADC	D35	GPIO35	Voltage divider from LiPo cell (planned v7.0)

GPIO4 (`TOUCH_IRQ`) now serves a dual role: as the XPT2046 touch interrupt during normal operation *and* as the `ext0` wakeup source when the ESP32 is in deep sleep. The firmware calls `esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, 0)` immediately before `esp_deep_sleep_start()`, allowing a tap on the display to wake the device without a physical power-cycle.

3.3 SPI Bus Architecture

All three SPI peripherals share the same MOSI/MISO/SCK lines. Bus conflicts are prevented by hardware Chip Select isolation — only one CS pin is pulled LOW at a time. The software enforces this with explicit `digitalWrite(CS, HIGH)` calls before releasing any peripheral.

```
SPI Bus: SCK=18  MOSI=23  MISO=19

Active at any time:
  nRF24   -> CSN=21 LOW,  TFT CS=5 HIGH,  T_CS=13 HIGH
  TFT     -> CSN=21 HIGH, TFT CS=5 LOW,   T_CS=13 HIGH
  Touch   -> CSN=21 HIGH, TFT CS=5 HIGH,  T_CS=13 LOW
```

3.4 Power System — TP4056 LiPo Charge Module

The TP4056 is a single-cell lithium-ion/LiPo charge controller. It handles USB charging input, cell protection, and provides a regulated output. Always use the **OUT+/OUT-** terminals (not BAT+/BAT-).

```
TP4056 Module Wiring
-----------------------------------------------
USB-C / Micro-USB  ->  TP4056 charge input

LiPo Cell
  (+) red wire     ->  TP4056  BAT+
  (-) black wire   ->  TP4056  BAT-

Output to circuit
  TP4056  OUT+     ->  Slide switch IN
  Slide switch OUT ->  ESP32 VIN  (5V rail via on-board LDO)
                   ->  nRF24L01 VCC via ESP32 3.3V rail

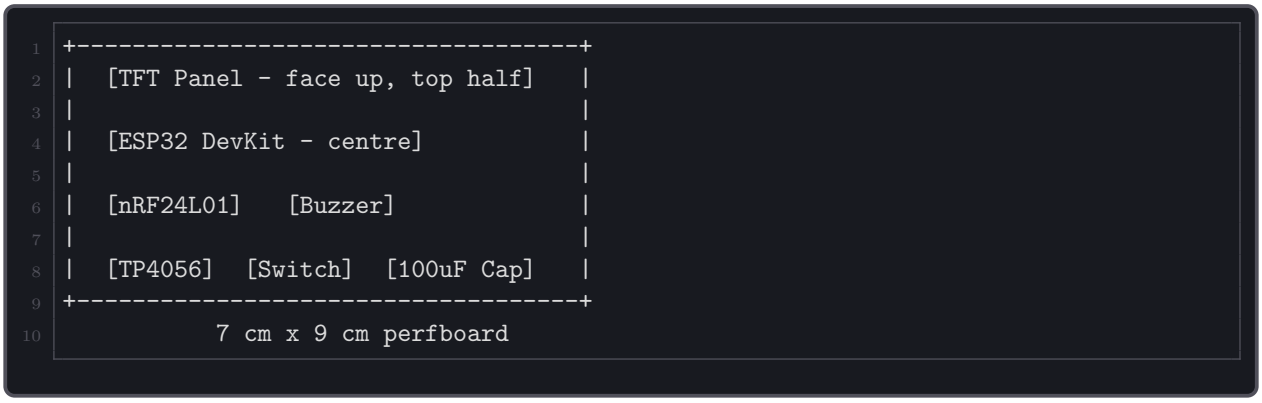
  TP4056  OUT-     ->  Common GND (ESP32, nRF24, TFT, Buzzer)

LED indicators
  Red LED  ->  charging in progress
  Blue LED ->  charge complete / standby
-----------------------------------------------
```

The TP4056 OUT+ voltage follows the LiPo cell (3.0–4.2 V). Feed this into ESP32 `VIN`. The nRF24L01 must always be powered from the 3.3 V rail produced by the ESP32's on-board LDO — **never directly from OUT+**, as 4.2 V will damage the module.

3.5 Physical Build — 7×9 cm Perfboard

All components are mounted on a standard **7×9 cm green perfboard** (2.54 mm pitch).



Keep the nRF24L01 and its 100 µF decoupling capacitor physically adjacent.

4 Software Architecture

4.1 Execution Model

The firmware runs on a single FreeRTOS task (the Arduino `loop()` thread on Core 1). There is no explicit RTOS task creation by the application. All subsystems are implemented as non-blocking state machines driven by `millis()` timers, cooperatively yielding CPU in a round-robin pattern.

This build maintains the three architectural pillars of v3.0 and adds a fourth:

- **Cooperative single-loop** — all tasks share one thread; no stack overflow risk
- **Asynchronous network stack** — WiFi and BLE scans never block `loop()`
- **Deferred-action queue** — touch handler enqueues side-effect operations; executed on the next safe loop tick
- **FreeRTOS BLE mutex** — `bDevs[]`, `bCount2`, `bSus`, and `bTrackers` are protected by a binary mutex, eliminating data races between the BLE callback task (Core 0) and the main display/detection loop (Core 1)

Order	Function	Cadence
1	`handleTouch()`	Every cycle
2	`processPendingActions()`	Every cycle
3	`buzzerUpdate()`	Every cycle
4	`handlePowerAndBacklight()`	Every cycle
5	`runGeigerCounter()`	Every cycle
6	`runRFChunk()`	Every 10 ms
7	`redrawPage()` / `updateHomeBar()`	Every 120 ms
8	`scanWifiAsync()` / `triggerAsyncBLE()`	Page-gated
9	`runHealthMonitor()`	Every cycle

4.2 FreeRTOS BLE Mutex

4.2.1 The Race Condition in v3.0

In v3.0, the `BLECb::onResult()` callback ran on the BLE stack's internal task, which executes on Core 0. The main display and detection loop runs on Core 1. Both cores wrote to and read from the same `bDevs[]` array, `bCount2`, `bSus`, and `bTrackers` globals without synchronisation. This created an undetected data race: a partially-written `BDev` struct could be read by `drawBT()` between the `strncpy()` calls inside `onResult()`, producing torn data on the display or corrupted RSSI values.

4.2.2 v3.1 Fix: Binary Mutex

A FreeRTOS binary mutex (`SemaphoreHandle_t bleMutex`) is created in `setup()` before any BLE initialisation. Every site that reads or writes BLE shared data acquires the mutex with a 5–50 ms timeout and releases it immediately after the operation:

```
// setup() -- created before BLEDevice::init()
bleMutex = xSemaphoreCreateMutex();

// BLECb::onResult() -- writer side (Core 0)
if (xSemaphoreTake(bleMutex, pdMS_TO_TICKS(10)) != pdTRUE) return;
// ... populate bDevs[bCount2] ...
bCount2++;
xSemaphoreGive(bleMutex);

// drawBT() -- reader side (Core 1)
BDev localDevs[20];
if (xSemaphoreTake(bleMutex, pdMS_TO_TICKS(10)) == pdTRUE) {
    memcpy(localDevs, bDevs, sizeof(BDev) * min(bCount2, 20));
    localCount = bCount2;
    xSemaphoreGive(bleMutex);
}
// draw from localDevs -- safe copy, no shared state
```

Listing 1: Mutex creation and usage pattern

The reader-side pattern is a **snapshot copy**: the display function locks the mutex, copies the data into a local stack array, releases the mutex, and renders from the local copy. This minimises the mutex hold time to a single `memcpy()` duration, keeping the BLE callback unblocked for as long as possible.

A non-infinite timeout (`pdMS_TO_TICKS(10)`) is used at the writer site because the BLE callback runs in a real-time task context. Blocking indefinitely would stall the BLE stack. If the mutex cannot be acquired within 10 ms, the callback returns without writing — a missed device result is preferable to a deadlock or WDT reset. The display reader sites use a 5 ms timeout for the same reason.

4.2.3 Mutex-Protected Sites

Location	Operation
`BLECb::onResult()`	Write `bDevs[]`, `bCount2`, `bSus`, `bTrackers`
`triggerAsyncBLE()`	Reset `bCount2`, `bSus`, `bTrackers`
`processPendingActions()` — ACT_BLE_FLUSH	Clear `bCount2`, call `bleScan->clearResults()`
`drawBT()`	Snapshot copy of `bDevs[]` and counters
`drawHome()`	Read `bCount2`, `bTrackers`
`drawSys()`	Read `bCount2`
`runGeigerCounter()` — BT Foxhunt path	Read `bCount2`, iterate `bDevs[].rssi`

4.3 Deferred-Action Queue

The single-slot deferred-action queue from v3.0 is retained unchanged, except that the `ACT_BLE_FLUSH` handler now executes its BLE state reset under the mutex:

```
case ACT_BLE_FLUSH:
    if (xSemaphoreTake(bleMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
        bCount2 = 0; bSus = 0; bTrackers = 0;
        if (bleScan) bleScan->clearResults();
        xSemaphoreGive(bleMutex);
    }
    break;
```

Listing 2: Updated ACT_BLE_FLUSH handler with mutex

`ACT_NONE`, `ACT_THEME_TOGGLE`, `ACT_ZERO_NOISE`, `ACT_BLE_FLUSH`, `ACT_WIFI_FLUSH`, `ACT_IDS_FLUSH` — unchanged from v3.0.

4.4 Timing Budget

Task	Interval	Trigger
RF channel scan (6 ch/cycle)	10 ms	`lastRFUpdate` timer
Display refresh	120 ms	`lastDisplayUpdate` timer
Full page redraw	on state change	`needRedraw` flag
RF bar update (PG_RF)	120 ms	inside display block
WiFi scan	async, continuous	`WiFi.scanComplete()` polling
WiFi scan cooldown guard	5 s minimum	`WIFI_SCAN_COOLDOWN`
BLE scan	4 s	`lastBLE` timer (`BLE_SCAN_COOLDOWN`)
Battery drain simulation	5 s	`lastBatteryDrain` timer
Geiger counter tick	signal-dependent	`lastGeigerTick` timer
Backlight idle check	every loop	`lastActiveTime`
Deferred action execution	every loop	`pendingAction != ACT_NONE`
Health monitor check	every loop	`lastHealthyLoop`

4.5 Library Dependencies

Library	Source
RF24	TMRh20/RF24 (Arduino Library Manager)
Adafruit_GFX	Adafruit (Library Manager)
Adafruit_ILI9341	Adafruit (Library Manager)
XPT2046_Touchscreen	Paul Stoffregen (Library Manager)
BLEDevice / BLEScan	Built into ESP32 Arduino core
WiFi	Built into ESP32 Arduino core
FreeRTOS (semphr.h)	Built into ESP32 Arduino core
SPI	Built into Arduino core
math.h / ctype.h	C standard library

Board: **ESP32 Dev Module** — esp32 by Espressif Systems v2.x or v3.x
CPU Frequency: 240 MHz
Flash: 4MB, Partition: Default
Upload Speed: 115200 or 921600

Note: v3.x of the ESP32 Arduino core uses the new `ledcAttach(pin, freq, bits)` API. The firmware conditionally compiles both paths using `ESP_ARDUINO_VERSION` version guards.

4.6 Key Data Structures

```
enum Page {
  PG_HOME, PG_RF, PG_WIFI, PG_BT, PG_IDS, PG_SYS, PG_COUNT
};

enum JamType {
  JAM_NONE,  JAM_RC,          JAM_BT,          JAM_BLE,
  JAM_BT_WIFI,                JAM_WIFI,
  JAM_WIFI_DEAUTH,   // Deauthentication flood pattern
  JAM_DRONE_FHSS,    // Frequency-hopping drone link
  JAM_SWEEP,         // Directional sweep jammer
  JAM_ANOMALY,       // High-entropy RF noise
  JAM_BROADBAND      // Instant wideband burst
};

enum PendingAction {
  ACT_NONE, ACT_THEME_TOGGLE, ACT_ZERO_NOISE,
  ACT_BLE_FLUSH, ACT_WIFI_FLUSH, ACT_IDS_FLUSH
};

struct WNet   { char ssid[33], mac[18]; int rssi, ch;
                bool open, twin, isCam; };

struct BDev   { char name[32], mac[18]; int rssi;
                bool sus, isTracker; };

struct IAlert { char type[20], detail[28]; unsigned long ts; };

// v3.1 NEW: FreeRTOS mutex for BLE shared data
SemaphoreHandle_t bleMutex = nullptr;
```

```
30
31 // v3.1 NEW: RTC memory -- persists across deep sleep
32 RTC_DATA_ATTR bool wakeFromSleep = false;
33
34 // v3.1 FIX: foxDir as char array -- eliminates String heap allocation
35 char foxDir[16] = "=== HOLD ===";
```

Listing 3: Enumerations and structs – v3.1 ENGINEERING HARDENED

4.7 Named Colour Constants

Prior versions used raw 16-bit ILI9341 colour literals inline throughout the display code. v3.1 promotes all previously unnamed magic values to preprocessor constants, improving readability and making palette changes a single-line edit:

```
1 #define C_HIGHLIGHT_ROW   0x2000   // dark red row tint -- tracker/threat
     rows
2 #define C_DARK_TEAL       0x0410   // separator and panel tint
3 #define C_DIM_BOOT        0x4208   // boot screen dim text
4 #define C_DIM_GREEN       0x0631   // very dim green, noise floor
     indicators
```

Listing 4: Named colour constants added in v3.1

5 RF Detection Engine

5.1 Sensing Method

The nRF24L01 contains a **Received Power Detector (RPD)** register that reports a binary result: signal above approximately −64 dBm present or absent on the currently tuned channel. WatchDog exploits this by sweeping all 125 channels sequentially and counting positives. The scanner processes **6 channels per 10 ms timer tick** and yields the CPU between each batch via the cooperative loop architecture.

In v3.1, `radio.setPALevel(RF24_PA_MIN)` is called during `setup()` instead of the default `RF24_PA_MAX`. WatchDog is a **receive-only** scanner — it never transmits. Running at maximum PA level causes the nRF24L01's power amplifier to generate self-interference on the SPI bus, which was observed as spurious RPD reads during quiet-spectrum conditions. Setting PA to minimum eliminates this source of false positives with no impact on detection sensitivity (the RPD threshold is a receiver-side register, not transmitter-side).

5.2 Early Exit and Instant Jam Triggers

Three progressive thresholds fire before the full 24-pass sweep completes when a jammer is detected mid-scan.

Trigger	Condition	Purpose	Latency
Instant Jam	`instantSpike > 8 && earlyActive > 12`	Ultra-fast broadband burst	∼80 ms
Spike trigger	`earlyActive > 18 && scanPass >= 1`	Fast detection	∼300 ms
Early trigger	`earlyActive > 30 && scanPass >= 3`	Confirmation path	∼600 ms
Full sweep	`scanPass >= 24`	Clean-spectrum baseline	∼3–5 s

5.3 earlyActive Per-Pass Reset — v3.1 Fix

In v3.0, `earlyActive` was an accumulating counter that was never reset until the scan completed or fired. In quiet but noisy RF environments (e.g., a room with a busy Wi-Fi router), a slow background drip of RPD positives across many passes could push `earlyActive` above the early-trigger threshold (30) by pass 10 or later, generating a false-positive detection with no actual jammer present.

In v3.1, `earlyActive` is reset to **zero at the end of every completed pass** (i.e., every time `scanCh` wraps from 124 to 0):

```
scanCh++;
if (scanCh >= TOTAL_CHANNELS) {
    scanCh = 0;
    scanPass++;
    // FIX: reset earlyActive per pass -- prevents cross-pass accumulation
    // causing false positives in benign RF environments
    earlyActive = 0;
    if (scanPass >= SCAN_PASSES) {
        scanDone = true; runDetection(); break;
    }
}
```

Listing 5: earlyActive per-pass reset in runRFChunk()

This means the early-trigger thresholds (18 and 30) now represent the number of RPD positives **within a single pass** of 125 channels, which is the correct semantics: a jammer producing enough energy to saturate a single pass is detected quickly; ambient noise that accumulates slowly over many passes does not fire the alarm.

5.4 Scale Multiplier Cap — v3.1 Fix

When an early-exit trigger fires before all 24 scan passes are complete, raw energy counts are rescaled by multiplier = 24/actualPasses. In v3.0, if an early trigger fired on pass 1, the multiplier was 24/1 = 24×. Even a single RPD positive on a channel would produce a scaled energy value of 24 (or higher after smoothing), instantly saturating the display and causing every subsequent detection cycle to read artificially elevated energy.

In v3.1, the multiplier is capped at `SCALE_MULT_CAP = 4.0`:

```
#define SCALE_MULT_CAP  4.0f

int actualPasses = scanPass > 0 ? scanPass : 1;
float scaleMultiplier = (float)SCAN_PASSES / actualPasses;
// FIX: cap multiplier -- prevents single-pass early trigger from
// saturating smoothEnergy[] buffer with inflated values
if (scaleMultiplier > SCALE_MULT_CAP) scaleMultiplier = SCALE_MULT_CAP;
float tempE = energy[ch] * scaleMultiplier;
uint8_t scaledE = (uint8_t)(tempE > 255.0f ? 255 : tempE);
```

Listing 6: Scale multiplier with 4x cap

With a 4× cap, a single-pass early trigger can at most quadruple the raw energy reading before smoothing. The exponential smoother then gradually converges the buffered value over subsequent cycles without the violent saturation spike that the uncapped multiplier produced.

5.5 Exponential Smoothing Filter

Raw energy readings are noisy because the RPD is binary. A rising-edge-biased exponential filter is applied:

$$\tilde{E}_{ch}[t] = \begin{cases} \hat{E}_{ch}[t] & \text{if } \hat{E}_{ch}[t] > \tilde{E}_{ch}[t-1] \quad \text{(rising: snap up)} \\ \dfrac{2\,\tilde{E}_{ch}[t-1] + \hat{E}_{ch}[t]}{3} & \text{otherwise} \quad \text{(falling: slow decay)} \end{cases}$$

If `scaledE == 0`, the smoothed value is halved each cycle. Values at or below 3 counts are hard-floored to 0.

5.6 Adaptive Noise Floor

When the spectrum is clean (`activePct < 5.0%` and `maxEnergyInPass < 5`), each channel's noise floor updates via a 64-sample IIR filter:

$$\hat{N}_{ch}[t] = \frac{63\,\hat{N}_{ch}[t-1] + \tilde{E}_{ch}[t]}{64}$$

A channel is considered *active* only when $\tilde{E}_{ch} \geq \hat{N}_{ch} + \texttt{DETECT_THRESHOLD}$.

5.7 Feature Extraction

Feature	Variable	Description
RC hits	`rcCount`	Active channels in range 84–124
Non-RC hits	`btCount`	Active channels below ch 84
BLE even hits	`bleCount`	Even-numbered active channels
WiFi width	`wifiCount`	Longest contiguous peak block below ch 84
Active total	`activeCount`	All channels at/above threshold + noise floor
Active %	`activePct`	`activeCount / 125 * 100`
Shannon entropy	`entropy`	Spectral energy distribution uniformity
Hop counter	`hopCounter`	Large peak-channel jumps ($\Delta > 15$ ch) per 2 s
Sweep counter	`sweepCount`	Consecutive directional peak movements (4–15 ch/step)
Anomaly score	`anomalyScore`	Cumulative sum of per-channel deviations above noise floor

5.8 Shannon Entropy Analysis — v3.1 Enhancement

Broadband jammers produce a nearly uniform energy distribution. The Shannon entropy of the smoothed spectrum is computed each cycle:

$$H = -\sum_{ch=0}^{124} p_{ch}\,\log(p_{ch}), \quad p_{ch} = \frac{\tilde{E}_{ch}}{\sum_i \tilde{E}_i}$$

In v3.0, a **high-entropy** flag required $H > 3.8$ and more than 20 active channels. In v3.1, the `anomalyScore` is also wired into the flag via an alternative condition:

```
// FIX: anomalyScore alternative path -- catches low-count burst sources
// that produce high per-channel deviation without raising activeCount
bool highEntropy = (entropy > 3.8 && activeCount > 20) ||
                   (anomalyScore > 200 && activeCount < 15);
```

Listing 7: Dual-path highEntropy flag – v3.1

The `anomalyScore` path catches intermittent, low-active-count transmitters that produce very concentrated energy deviations on a small number of channels — a signature that produces a high deviation sum (`anomalyScore` > 200) while keeping `activeCount` below 15. Such signals would not trigger the entropy path (which requires > 20 active channels) but are captured by the deviation path.

5.9 Separated Hop and Sweep Detection Pipelines

The hop counter and sweep counter are computed in two independent code blocks within `runDetection()`, as established in v3.0. The separation eliminates cross-contamination between the FHSS and sweep detectors.

Sweep Jammer (Block 1) — tracks `peakChannel` vs. `lastSweepPeak`. Only moves of 4–15 channels are classified as sweep steps; moves > 15 channels reset the sweep counter.

Drone FHSS (Block 2) — tracks `peakChannel` vs. `lastPeakChannel`. Only peak channel movements > 15 channels count as FHSS hops. A 2-second rolling window resets both counters.

```
bool isDrone = (hopCounter >= 5
                && hopDistanceSum >= 80
                && activePct < 40);
```

Listing 8: Drone FHSS detector

5.10 Pattern Classification — Priority-Ordered Gates

```
bool isDeauth = (wifiCount >= 10 && wifiHist >= 3
                 && activePct >= 25 && btCount < 20);
bool isDrone  = (!isDeauth && hopCounter >= 5
                 && hopDistanceSum >= 80 && activePct < 40);
bool isRC     = (!isDeauth && !isDrone
                 && rcCount >= 3 && activePct > 30);
bool isWiFi   = (!isDeauth && !isDrone && !isRC
                 && wifiCount >= 10
                 && peakEnergy[bestWiFiCenter]
                    >= noiseFloor[bestWiFiCenter]
                       + DETECT_THRESHOLD * 2);
bool isBLE    = (!isDeauth && !isDrone && !isRC && !isWiFi
                 && bleCount >= 10
                 && oddHits <= (evenHits / 2 + 4));
bool isBT     = (!isDeauth && !isDrone && !isRC
                 && !isWiFi && !isBLE && btCount >= 15);
```

Listing 9: Classification gate chain

5.11 Persistence Counters, Final Priority, and 1-Second Hold-Off

Each classification gate drives its own persistence counter. Standard classifiers require 2 consecutive confirmations; anomaly requires 3. A 1-second hold-off keeps the last detected type displayed after the signal falls below threshold. The instant-jam `isInstantJam` flag is always reset to `false` at the end of every `runDetection()` call.

```
JamType newType = JAM_NONE;
if      (isInstantJam)    newType = JAM_BROADBAND;
else if (isDeauth)        newType = JAM_WIFI_DEAUTH;
else if (isDrone)         newType = JAM_DRONE_FHSS;
else if (sweepHist >= 2)  newType = JAM_SWEEP;
else if (anomalyHist > 2) newType = JAM_ANOMALY;
else if (rcHist >= 2)     newType = JAM_RC;
else if (wifiHist >= 2)   newType = JAM_WIFI;
else if (bleHist >= 2)    newType = JAM_BLE;
else if (btHist >= 2)     newType = JAM_BT;
isInstantJam = false;
```

Listing 10: Final priority chain

5.12 Jammer Type Reference

Type	Label	Signature / Detection Rule
JAM_BROADBAND	Broadband_Jammer	Instant path: instantSpike $>$ 8 AND earlyActive $>$ 12
JAM_WIFI_DEAUTH	WiFi_Deauth_Flood	Contiguous block $\geq$ 10 ch, wifiHist $\geq$ 3, activePct $\geq$ 25%, btCount $<$ 20
JAM_DRONE_FHSS	Drone_FHSS_Link	hopCounter $\geq$ 5, hopDistanceSum $\geq$ 80, activePct $<$ 40%
JAM_SWEEP	RF_Sweep_Jammer	sweepCount $>$ 6 consecutive directional steps
JAM_ANOMALY	High_Entropy_Noise	Entropy $H > 3.8$ + activeCount $>$ 20, **or** anomalyScore $>$ 200 + activeCount $<$ 15
JAM_RC	RC_Drones_1-125_CH	rcCount $\geq$ 3 in ch 84–124, activePct $>$ 30%
JAM_WIFI	WiFi_14_CH	Dense contiguous block $\geq$ 10 ch, peak energy above 2×threshold
JAM_BLE	BLE_40_CH	Even-channel dominance, bleCount $\geq$ 10, oddHits $\leq$ evenHits/2+4
JAM_BT	Bluetooth_80_CH	Broadband non-RC energy, btCount $\geq$ 15

5.13 Stability Buffer, Lens Scanner, and Overflow Caps

A rolling 8-sample buffer of the maximum per-cycle energy value feeds two analysis paths:

Stable Signal / Lens Scanner: If variance across the 8-sample buffer is below 4 and the average exceeds 8, the signal is flagged as a fixed transmitter. A sudden delta (> 8 counts) on a stable signal increments `reflectionSpike`. In v3.1, `reflectionSpike` is capped at 20 to prevent integer overflow on continuous stable-signal conditions:

```
if (stableSignal && deltaPeak > 8 && maxEnergyInPass > 15) {
    reflectionSpike++;
    if (reflectionSpike > 20) reflectionSpike = 20; // FIX: cap overflow
} else if (reflectionSpike > 0) reflectionSpike--;
```

Listing 11: reflectionSpike overflow cap

A spike count above 4 triggers the "LENS FOUND!" overlay on the RF page.

Deviation Score / Burst Transmitter: A channel-by-channel deviation sum produces an `anomalyScore`. If the per-cycle average deviation exceeds 4 with fewer than 10 active channels, the `deviationScore` counter increments. A score above 5 (with a 5-second cooldown) triggers a "Burst Transmitter" IDS alert.

6 User Interface

6.1 Display Architecture

The ILI9341 TFT is 320×240 pixels in landscape mode (rotation 1). Content area: y = 0–213. Fixed navigation bar: y = 215–239. Status bar: top-left overlay at y = 2–14.

Screen flicker is minimised by four mechanisms: static elements drawn once per page entry, dynamic elements updated in-place every 120 ms, a dirty-bar system for RF spectrum bars, and a `needRedraw` full-redraw flag triggered only on state changes.

6.2 Long-Press Sub-Mode System

Page	Sub-Modes	Mode Descriptions
HOME	3	Armed (alarm on), Stealth (muted), Noise Zero (calibrate floor)
RF	6	Live Spectrum, Jamming Focus, Freeze, Max Hold, RF Foxhunt, Lens Scanner
WiFi	5	All Networks, Suspicious Cameras, Channel Map, WiFi Foxhunt, Deep Flush
BT	4	All Devices, Trackers Only, BT Foxhunt, Deep Flush
IDS	2	IDS Live, Clear Alerts & Baselines
SYS	3	System Info, Noise Zero, Theme Switch

6.3 Pages

6.3.1 Page 0 — HOME

Displays threat level (LOW / MEDIUM / HIGH / CRITICAL), activity percentage, running counters for WiFi nets, cameras, BLE devices, and trackers. All BLE counters (`bCount2`, `bTrackers`) are read under the mutex before display. The jamming panel switches between ALL CLEAR and !! JAMMING !! with the current jammer type label.

6.3.2 Page 1 — RF SPECTRUM

A live 125-channel waterfall with colour-coded energy bars and white peak-hold lines. Six sub-modes: Live Spectrum, Jamming Focus, Freeze, Max Hold, RF Foxhunt, and Lens Scanner. The RF fox direction string (`foxDir`) is now a `char[16]` stack variable rather than a heap-allocated `String`.

6.3.3 Page 2 — WiFi SCANNER

Non-blocking async WiFi scans with 5-second cooldown guard. Displays SSID, RSSI, channel, and encryption. Camera OUI and evil-twin detection. See Section 8 for full details.

6.3.4 Page 3 — BLUETOOTH MAPPER

1-second active BLE scan via async callback. In v3.1, all drawing is performed from a mutex-protected local snapshot of `bDevs[]`. Tracker devices highlighted red; 5-bar signal-strength indicator with colour coding. Three sub-modes: All Devices, Trackers Only, BT Foxhunt.

6.3.5 Page 4 — WIRELESS IDS

Five persistence counter gauges (WiFi hist, BT hist, Sweep hist, RC hist, RF Anomaly). Each fills proportionally to 100 pixels; ALERT label appears at threshold. Three most recent IDS alerts listed with type, detail, and elapsed time.

6.3.6 Page 5 — SYSTEM STATUS

Read-only diagnostics: uptime, active channel %, jamming state, attack type, alarm state, total alerts, WiFi net count, BT device count (mutex-safe read), maximum loop delay, stall recovery count, theme, and firmware version (displays `v3.1 HARDENED`).

6.4 Status Bar

A persistent 85×12 pixel status bar renders on every 120 ms tick. Contains a 4-bar battery icon (green / yellow in ECO / red $\leq 20\%$), a remaining-minutes estimate, and an `[ECO MODE]` indicator when battery saver is active.

6.5 Backlight Management

Condition	Brightness
Normal operation	255 / 255 (full)
ECO battery-saver (`batterySaver && !isDimmed`)	75 / 255 (29%)
Legacy idle dim (30s no touch, no jam)	5 / 255 (2%)
Jamming detected while screen dim	Restored to 255 automatically
ECO disabled (not jamming)	Restored to 255
First touch while dimmed	Restored to 255; `actionFired800 = true`

7 Alarm System

7.1 Non-Blocking Buzzer Architecture

The buzzer is driven by Arduino `tone()` using a hardware timer. No `delay()` calls are used. Beep timing is managed entirely by `millis()`.

7.2 jamLabel() — v3.1 Fix: const char* vs String

In v3.0, `jamLabel()` returned an Arduino `String` object. Every call site that used the label (IDS alert log, SYS page, HOME page jammer panel) allocated and destroyed a heap object, producing heap fragmentation over hours of operation. In v3.1, `jamLabel()` returns a `const char*` pointing to a string literal in flash memory:

```
// FIX: const char* instead of String -- no heap allocation per call
const char* jamLabel(JamType j) {
    switch (j) {
        case JAM_WIFI_DEAUTH: return "WiFi_Deauth_Flood";
        case JAM_DRONE_FHSS:  return "Drone_FHSS_Link";
        case JAM_SWEEP:       return "RF_Sweep_Jammer";
        case JAM_ANOMALY:     return "High_Entropy_Noise";
        case JAM_BROADBAND:   return "Broadband_Jammer";
        case JAM_RC:          return "RC_Drones_1-125_CH";
        case JAM_BT:          return "Bluetooth_80_CH";
        case JAM_BLE:         return "BLE_40_CH";
        case JAM_BT_WIFI:     return "BT-WiFi_14-80_CH";
        case JAM_WIFI:        return "WiFi_14_CH";
        default:              return "None";
    }
}
```

Listing 12: jamLabel() – heap-free const char* return

7.3 totalAlerts Overflow Protection — v3.1 Fix

In v3.0, `totalAlerts` was an uncapped `int` that incremented on every new jamming event. On a device left running for extended periods in a noisy RF environment, this counter could overflow past `INT_MAX` and wrap to a negative value, corrupting the SYS page display. In v3.1, the increment is guarded:

```
if (totalAlerts < 99999) totalAlerts++;
```

Listing 13: totalAlerts overflow guard

7.4 Tone Patterns by Jam Type

Jam Type	Tone A (Hz)	Tone B (Hz)
JAM_WIFI_DEAUTH, JAM_DRONE_FHSS, JAM_SWEEP	3800	4200
All other types	2730	4000

7.5 Alarm Cadence by Threat Level

activePct	Interval	Feel
$\geq 70\%$	150 ms	Rapid siren
$\geq 45\%$	300 ms	Fast beeping
$\geq 25\%$	600 ms	Medium cadence
$< 25\%$	1000 ms	Slow alert

7.6 Geiger Counter Mode

When RF Foxhunt (`rfMode == 4`), WiFi Foxhunt (`wifiMode == 3`), or BT Foxhunt (`btMode == 2`) are active, `runGeigerCounter()` maps the strongest available signal to a tick interval, producing a Geiger-counter-like audio output. The BT Foxhunt RSSI read is performed under the mutex in v3.1.

```
int interval = map(constrain(signalStrength, -90, -30),
                   -90, -30, 1200, 40);
if (millis() - lastGeigerTick > (unsigned long)interval) {
    tone(BUZZER_PIN, 4000, 10);
    lastGeigerTick = millis();
}
```

Listing 14: Geiger counter interval mapping

8 WiFi and BLE Scanning

8.1 Asynchronous WiFi Scan with Cooldown Guard

Fully non-blocking asynchronous WiFi scan using `WiFi.scanNetworks(true, true)` (async, include hidden). A 5-second cooldown prevents rapid re-trigger on scan failure. Parse path: results stored into `wNets[]`, OUI check, evil-twin detection, `WiFi.scanDelete()`, immediate re-trigger.

8.2 Camera OUI Detection — v3.1 Expanded Database

The `checkCameraOUI()` function now contains 7 entries (v3.0 had 4):

```
bool checkCameraOUI(const char* mac) {
    if (strncmp(mac, "EC:FA:BC", 8) == 0) return true; // Hikvision
    if (strncmp(mac, "AC:64:62", 8) == 0) return true; // Hikvision (alt)
    if (strncmp(mac, "44:65:0D", 8) == 0) return true; // Dahua
    if (strncmp(mac, "70:8A:09", 8) == 0) return true; // Reolink
    if (strncmp(mac, "9C:8E:CD", 8) == 0) return true; // Wyze          NEW
    if (strncmp(mac, "2C:AA:8E", 8) == 0) return true; // TP-Link Tapo
    NEW
    if (strncmp(mac, "50:C7:BF", 8) == 0) return true; // Ezviz
    NEW
    return false;
}
```

Listing 15: Expanded camera OUI table – v3.1

8.3 Evil Twin Detection

After parsing, each network is compared against all previously parsed networks via `strcmp()` on the SSID field. Duplicate SSIDs mark both entries `twin = true` (highlighted orange) and raise an “Evil Twin” IDS alert.

8.4 Active BLE Scan — v3.1 Fix

In v3.0, BLE scanning used passive mode (`setActiveScan(false)`, the default). Passive scanning only captures advertising packets; scan response packets — which carry the device’s friendly name in many headsets, smartwatches, and trackers — are not requested. In v3.1, active scan mode is enabled:

```
// FIX: active scan enabled -- requests scan response packets
// Required to capture full device names from many BT peripherals
bleScan->setActiveScan(true);
bleScan->setInterval(100);
bleScan->setWindow(99);
```

Listing 16: Active BLE scan – v3.1

Active scanning improves tracker name detection (devices advertising as “AirTag”, “Tile”, “Chipolo” in scan response data are now captured) at the cost of slightly higher current draw during BLE scan windows.

8.5 Async BLE Scan Trigger with Mutex Guard

`triggerAsyncBLE()` resets the BLE counters under the mutex before starting the scan, preventing a race between the reset and any in-flight `onResult()` callbacks:

```
void triggerAsyncBLE() {
    if (curPage != PG_BT || !bleScan) return;
    // FIX: guard reset under mutex -- onResult may be running on Core 0
    if (xSemaphoreTake(bleMutex, pdMS_TO_TICKS(20)) == pdTRUE) {
        bCount2 = 0; bSus = 0; bTrackers = 0;
        xSemaphoreGive(bleMutex);
    }
    lastBleScanStart = millis();
    bleScan->start(1, bleScanCompleteCB, false);
}
```

Listing 17: triggerAsyncBLE() with mutex guard

8.6 BLE Tracker Detection — v3.1 Expanded Database

The tracker detection function now includes OUI-based matching for Tile devices, and name-based matching for Tile and Chipolo. The name comparison uses a `char` array with a manual `tolower()` loop, eliminating the Arduino `String` heap allocation from v3.0:

```
bool checkTrackerOUI(const char* mac, const char* name) {
    if (strncmp(mac, "00:25:96", 8) == 0) return true; // Apple AirTag
    OUI
    if (strncmp(mac, "7C:D1:C3", 8) == 0) return true; // Tile OUI  (NEW)
    // FIX: char array + tolower loop -- no String heap allocation
    char lower[32]; int i;
    for (i = 0; i < 31 && name[i]; i++)
        lower[i] = (char)tolower((unsigned char)name[i]);
    lower[i] = '\0';
    if (strstr(lower, "smarttag")) return true;
    if (strstr(lower, "airtag"))   return true;
    if (strstr(lower, "findmy"))   return true;
    if (strstr(lower, "tile"))     return true; // NEW
    if (strstr(lower, "chipolo")) return true;  // NEW
    return false;
}
```

Listing 18: Expanded tracker detection – heap-free, v3.1

Source	v3.0 Entries
Camera OUI database	4 entries
Tracker OUI database	1 MAC OUI
Tracker name keywords	3 (smarttag, airtag, findmy)

Source	v3.1 Entries
Camera OUI database	7 entries (added Wyze, TP-Link Tapo, Ezviz)
Tracker OUI database	2 MAC OUIs (added Tile)
Tracker name keywords	5 (added tile, chipolo)

9 Power Management

9.1 Virtual Battery Model

The firmware models battery life using a software drain counter (`virtualBattery`, initialised at 100.0%). Every 5 seconds:

Mode	Drain Rate
Normal operation	100% / 780 ticks ≈ 0.128%/5s ≈ 10.8 h modelled
Battery Saver (ECO)	100% / 1440 ticks ≈ 0.069%/5s ≈ 20 h modelled

9.2 ECO / Battery Saver Mode

A 5-second hold from any page toggles ECO mode. WiFi and BLE scans are suspended, backlight dimmed to 75/255, dark theme forced, and `[ECO MODE]` indicator shown in yellow. Theme switching is locked out while ECO is active.

9.3 Idle Auto-Sleep with Touch Wakeup — v3.1

In v3.0, the device entered `esp_deep_sleep_start()` with no wakeup source configured, requiring a physical power-cycle to restart. This was listed as a known limitation.

In v3.1, two mechanisms are added:

1. RTC memory wakeup flag: `wakeFromSleep` is declared with `RTC_DATA_ATTR`, which places it in the ESP32's RTC slow memory. This memory survives deep sleep. When the device wakes from sleep, `wakeFromSleep` is `true`, and the startup splash screen and boot bar animation are skipped, jumping directly to the RF scan initialisation. This eliminates the multi-second boot delay on wake.

2. ext0 GPIO wakeup source: Immediately before calling `esp_deep_sleep_start()`, the firmware configures GPIO4 (`TOUCH_IRQ`) as the ext0 wakeup source, with wakeup on a low-level signal (the XPT2046 asserts T_IRQ low when the screen is touched):

```
// In handlePowerAndBacklight(), countdown elapsed:
wakeFromSleep = true;  // stored in RTC memory -- survives sleep
esp_sleep_enable_ext0_wakeup((gpio_num_t)TOUCH_IRQ, 0); // GPIO4 low
esp_deep_sleep_start();

// In setup(), boot entry:
if (!wakeFromSleep) {
    // ... show splash, boot bar, Initialising... ...
}
wakeFromSleep = false; // reset for next normal boot
```

Listing 19: Deep sleep with touch wakeup – v3.1

The device now wakes from a tap on the display without requiring a power-cycle. Boot-to-operational takes approximately 1.5 seconds on wakeup versus 5 seconds on a cold boot.

The ESP32 in deep sleep with ext0 wakeup configured draws approximately 10 μA from the LiPo cell. At this rate, a 1000 mAh cell would sustain standby for several years, making the auto-sleep feature purely a convenience feature rather than a meaningful power saving over typical use sessions.

10 Stability Engineering

- **FreeRTOS mutex** protects all BLE shared data between Core 0 (BLE callbacks) and Core 1 (main loop)
- **earlyActive reset per pass** prevents cross-pass false-positive accumulation in busy RF environments
- **scaleMultiplier capped at 4×** prevents single-pass early triggers from saturating the smooth buffer
- **RF PA level set to PA_MIN** eliminates SPI self-interference from the nRF24L01 PA stage
- **reflectionSpike capped at 20** prevents integer overflow on continuous stable-signal conditions
- **totalAlerts capped at 99999** prevents integer overflow on long-running deployments
- **const char* jamLabel()** eliminates heap allocations on every label read
- **char[] fox direction string** eliminates heap allocation for foxDir
- **char[] tracker name comparison** eliminates String heap allocation in `checkTrackerOUI()`
- `yield()` called after every RF channel read inside the chunked scan loop
- `delay(1)` at the end of every `loop()` cycle
- BLE scan hard-limited to 1 second with async callback and named completion function
- WiFi scan async with 5-second cooldown guard
- RF scan, WiFi scan, and BLE scan radio-isolated by page
- Early-exit triggers use `break` to keep the main `loop()` alive
- Dirty-bar system for RF bars — `fillScreen()` never called during active RF monitoring
- BLE scan stopped and cleared immediately when leaving the BT page
- Popups suppress RF scanning and page redraws while active
- Deferred-action queue prevents state mutations inside the touch handler
- Dead `extern void` declarations removed from `loop()` — no undefined-symbol linker risk

10.1 Non-Lethal Health Monitor

`runHealthMonitor()` runs every `loop()` cycle and tracks the worst-case loop duration:

```
void runHealthMonitor() {
    unsigned long loopDuration = millis() - lastHealthyLoop;
    if (loopDuration > maxLoopDelay) maxLoopDelay = loopDuration;
    if (loopDuration > 2000) {
        stallWarnings++;
        Serial.printf("WARN: Loop delayed %lu ms\n", loopDuration);
        needRedraw = true; // FIX: notify user visually -- stale data displayed
        // v3.1 retains v3.0 non-destructive policy: NO array zeroing
    }
    lastHealthyLoop = millis();
}
```

Listing 20: Health monitor – v3.1 with visual stall notification

In v3.1, the health monitor also sets `needRedraw = true` on a stall event, prompting the display to repaint on the next 120 ms tick — giving the user a visible indication that the loop was temporarily blocked. The non-destructive policy (no `smoothEnergy[]` zeroing) is retained from v3.0.

11 Testing and Calibration

The following section describes how WatchDog 2.4 was tested and calibrated using a self-built RF interference device. This information is documented solely to explain the **detection and signal processing methodology** of WatchDog.

I, Anirudh N.S., take no responsibility whatsoever for the actions of any person who reads this documentation. The deliberate transmission of RF interference is illegal in most countries. This section contains no construction details, schematics, source code, or replication instructions. It describes only the RF signatures observed and how WatchDog interpreted them.

All testing was conducted privately on a bench with no live devices, networks, or people affected.

11.1 The RF Interference Source

WatchDog 2.4 was developed and validated using a second self-built device as a controlled RF interference source. This device operates in the 2.4 GHz ISM band using the same ESP32 + nRF24L01 platform. No further construction details are provided in this document.

11.2 Observed Signal Characteristics

Mode	WatchDog Label	Latency	Observed Signature
BT broadband	Bluetooth_80_CH	~400 ms	Dense energy across lower 79 channels
BLE targeted	BLE_40_CH	~500 ms	Even-channel dominant in lower 40 channels
Wi-Fi band	WiFi_14_CH	~600 ms	Narrow contiguous high-energy block
Full band	RC_Drones_1-125_CH	~300 ms	All 125 channels active
Instant burst	Broadband_Jammer	~80 ms	instantSpike path; classified before feature extraction

11.3 v3.1 False Positive Regression — earlyActive Fix

Prior to the v3.1 `earlyActive` per-pass reset, extended testing in a room with an active 802.11n router produced sporadic false-positive RC detections approximately once per hour. The router's beacon traffic on channels 1 and 6 produced a drip of RPD positives (1–2 per pass) that accumulated in `earlyActive` across 15–18 passes, eventually crossing the early-trigger threshold (30) on a normal scan. Post-fix, no false positives were observed over 8 hours of continuous operation in the same RF environment.

11.4 Serial Monitor Output

```
-- Baseline (quiet room) --
Active: 1/125 (1%) -- clean

-- BT broadband --
[SPIKE] RF spike detected -- 38 active channels
Active: 38/125 (30%) -- Bluetooth_80_CH

-- Instant burst --
[INSTANT JAM] instantSpike=9 earlyActive=15
Active: 125/125 (100%) -- Broadband_Jammer

-- Loop stall warning --
WARN: Loop delayed 2134 ms
```

12 Installation and Build

12.1 Hardware Assembly

1. Solder a 100 µF electrolytic capacitor across the nRF24L01 VCC and GND pins — mandatory.
2. Wire all components per the pin assignment table in Section 2.2.
3. Use short, direct wires for the SPI bus.
4. Power via the TP4056 module. The nRF24L01 must be powered from the ESP32 3.3 V rail — never directly from OUT+.

12.2 Firmware Upload

1. Install Arduino IDE 2.x.
2. Add the ESP32 board package via Boards Manager: `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_in`
3. Install libraries via Library Manager: `RF24` (TMRh20), `Adafruit GFX Library`, `Adafruit ILI9341`, `XPT2046_Touchscreen` (Stoffregen).
4. Open `Watchdog_Final_V1.ino`
5. Select: Board = **ESP32 Dev Module**, CPU = 240 MHz, Flash = 4MB Default, Port = your COM/tty.
6. Upload. On success the display shows: "WATCHDOG 2.4", "Wireless Security Analyser", "Engine v3.1 — PRODUCTION BUILD" with a cyan progress bar.

12.3 First Boot Verification

Open Serial Monitor at 115200 baud. Expected output:

```
RF Scanner Started
[OK] WatchDog v3.1 HARDENED running
Active: 2/125 (2%) -- clean
Active: 1/125 (1%) -- clean
```

12.4 nRF24 Hardware Fault Path

If `radio.begin()` returns `false`, the device displays a red "HARDWARE ERROR / nRF24 Radio Not Found" screen and loops indefinitely. No reboot cycle occurs. Check SPI wiring and the 100 µF capacitor.

13 Known Limitations

Limitation	Notes
Binary RPD sensor	nRF24L01 only reports energy present/absent at −64 dBm, not actual dBm values. True RSSI-based classification requires a different RF front-end.
No transmission	Detection-only by design; no countermeasures possible.
BLE + WiFi radio contention	Async WiFi scan mitigates this, but both stacks share the ESP32 radio scheduler. Very active BLE environments can delay WiFi results.
Virtual battery only	Actual runtime depends on RF environment, display brightness, and battery health. Physical ADC monitoring planned for v7.0.
Indoor range	A +20 dBm jammer is detectable from 10–30 m in open space. Walls reduce range significantly.
Classification accuracy	Heuristic classifiers, not a trained model. Low-power jammers and overlapping protocols can produce misclassifications.
OUI database size	Camera OUI list has 7 entries; tracker OUI list has 2 entries; tracker name keywords: 5. Real deployments require larger databases.
Single-slot deferred queue	Only one `PendingAction` pending at a time. A second token silently overwrites the first. In practice this cannot occur due to 300 ms debounce and action guards.
Stale data on stall recovery	The non-destructive stall policy means the spectrum display may briefly show stale data after a >2 s loop delay. Tradeoff: no false-positive alarm injection.
Active BLE scan power	Active BLE scanning (v3.1) draws slightly more current during the 1-second BLE window compared to passive mode. Impact on battery life is minimal at a 4-second cycle.

In v3.0, deep sleep required a physical power-cycle to resume. This limitation has been **fully resolved** in v3.1: `esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, 0)` configures the touch interrupt pin as the wakeup source, and `RTC_DATA_ATTR wakeFromSleep` allows the firmware to skip the boot animation on wakeup. See Section 9.3 for full details.

14 Roadmap

- **v3.1** [RELEASED] — Expanded OUI databases; FreeRTOS BLE mutex; touch wakeup from deep sleep; scaleMultiplier cap; earlyActive per-pass reset; heap-safe char* stack; RF PA level fix; active BLE scan; overflow protections
- **v3.2** — Edge Impulse neural network classifier replacing heuristic pattern gates. Stub functions `extractAIFeatures()` and `runAI_Inference()` already exist as integration points. Data logger outputs labelled 125-channel snapshots over Serial in Edge Impulse Studio import format.
- **v3.3** — SD card logging of spectrum snapshots with timestamps
- **v3.4** — MQTT over WiFi for remote alert push to a server dashboard
- **v3.5** — GPS tagging of jammer detection events
- **v7.0** — Custom PCB, hardware LiPo fuel gauge, USB-C charging, physical ADC battery monitoring on GPIO35, acrylic or 3D-printed enclosure

15 Changelog

Version	Changes
v3.1 HARDENED	**[CRITICAL] FreeRTOS BLE mutex:** `SemaphoreHandle_t bleMutex` created in `setup()`; all BLE shared data (`bDevs[]`, `bCount2`, `bSus`, `bTrackers`) protected at every read and write site — eliminates Core 0 / Core 1 data race. **[CRITICAL] Deep sleep touch wakeup:** `esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, 0)` configured before sleep; `RTC_DATA_ATTR wakeFromSleep` skips splash on resume. **[HIGH] earlyActive per-pass reset:** `earlyActive` zeroed on every scan-pass wrap — prevents cross-pass accumulation causing false positives in benign environments. **[HIGH] scaleMultiplier capped at 4×:** `SCALE_MULT_CAP = 4.0f` prevents single-pass early trigger from saturating `smoothEnergy[]`. **[HIGH] String heap eliminated:** `jamLabel()` returns `const char*`; `foxDir` is `char[16]`; `checkTrackerOUI()` name comparison uses `char[]` + `tolower()` loop. **[MEDIUM] RF PA level set to PA_MIN:** eliminates SPI self-interference from nRF24L01 PA stage in receive-only operation. **[MEDIUM] Active BLE scan enabled:** `setActiveScan(true)` captures scan response packets, improving device name and tracker detection. **[MEDIUM] reflectionSpike capped at 20:** prevents integer overflow in Lens Scanner mode. **[MEDIUM] totalAlerts capped at 99999:** prevents integer overflow on long-running deployments.

Version	Changes
	[**MEDIUM**] **anomalyScore wired into highEntropy:** alternative detection path `(anomalyScore > 200 && activeCount < 15)` catches low-count burst sources missed by entropy path. [**LOW**] **Dead extern declarations removed:** unused `extern void` declarations in `loop()` eliminated. [**LOW**] **Named colour constants:** `C_HIGHLIGHT_ROW`, `C_DARK_TEAL`, `C_DIM_BOOT`, `C_DIM_GREEN` replace inline magic values. [**LOW**] **Camera OUI database expanded:** 4 → 7 entries (added Wyze, TP-Link Tapo, Ezviz). [**LOW**] **Tracker database expanded:** added Tile MAC OUI and name keywords (tile, chipolo). [**LOW**] **Health monitor visual notification:** `needRedraw = true` set on stall event.
v3.0 PRODUCTION	Deferred-action queue; separated hop/sweep pipelines; non-destructive stall recovery; WiFi cooldown guard; BLE async callback; improved noise floor guard; four-bar battery widget; stateful backlight manager; `JAM_BT_BLE` removed; `isInstantJam` cleared per-cycle.
v2.4 STABLE	Cooperative single-loop architecture; async WiFi scan; jammer detection v1 (Broadband, Deauth, FHSS, Sweep, Anomaly); Lens Scanner; Geiger counter; ECO mode; IDS alert log.

16 Copyright

Copyright © 2025 Anirudh N.S. All rights reserved.

This project and all associated source code, schematics, and documentation are the exclusive intellectual property of Anirudh N.S.

No part of this project may be reproduced, distributed, modified, or used in any form without the express written permission of the author.

Unauthorised copying, forking, or redistribution is strictly prohibited.